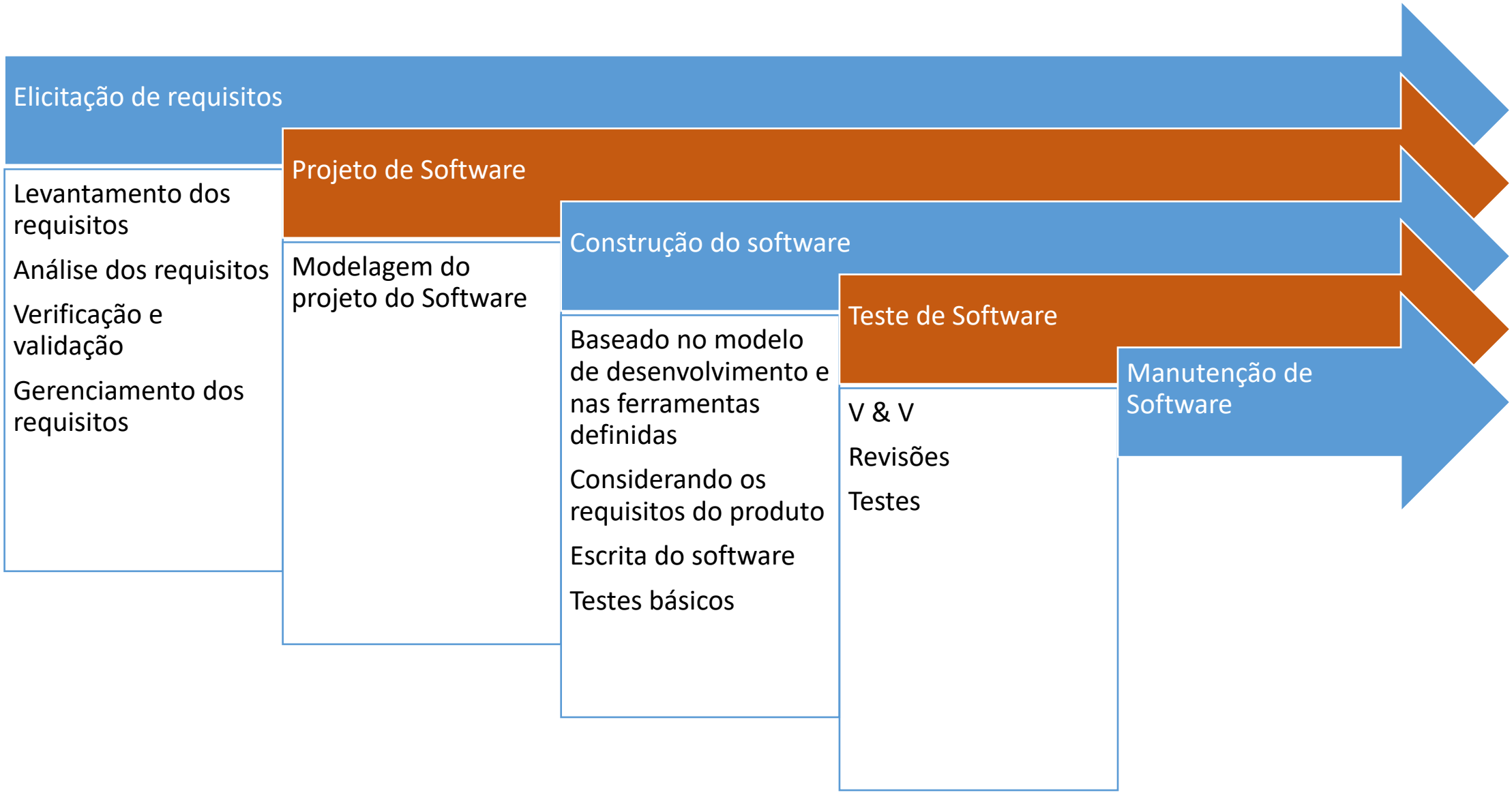
Engenharia de Software

Manutenção de Software

Profa. Patricia Rucker de Bassi

Engenharia de Software



Motivação

- Visão das organizações:
  - Globalização
  - Concorrência
  - Manter-se competitiva

- Consequências na Engenharia de software:
  - Mudanças nas regras de negócio
  - Mudanças nas estratégias organizacionais
  - Inserção de novas tecnologias

Conceito

- A atividade de manutenção de software é caracterizada pela modificação de um produto de software já entregue ao cliente, para correção de eventuais erros, melhora em seu desempenho ou qualquer outro atributo, ou ainda para adaptação desse produto a um ambiente modificado.

(IEEE, 2004)

- Normalmente é a mais longa do ciclo de vida do produto de software.
- Produtos de software podem mudar devido a ações corretivas e não corretivas de software.
- Pode ocorrer modificação em componentes existes e/ou desenvolvimento de novos componentes.

(Sommerville, 2011)

Necessidades – Swebok

- Corrigir falhas
- Melhorar o design/projeto
- Implementar melhorias
- Interface com outros softwares
- Adaptar os programas:
  - Hardware
  - Software
  - Recursos do sistema
  - Instalações de telecomunicação
- Migrar o software legado
- Desativar o software

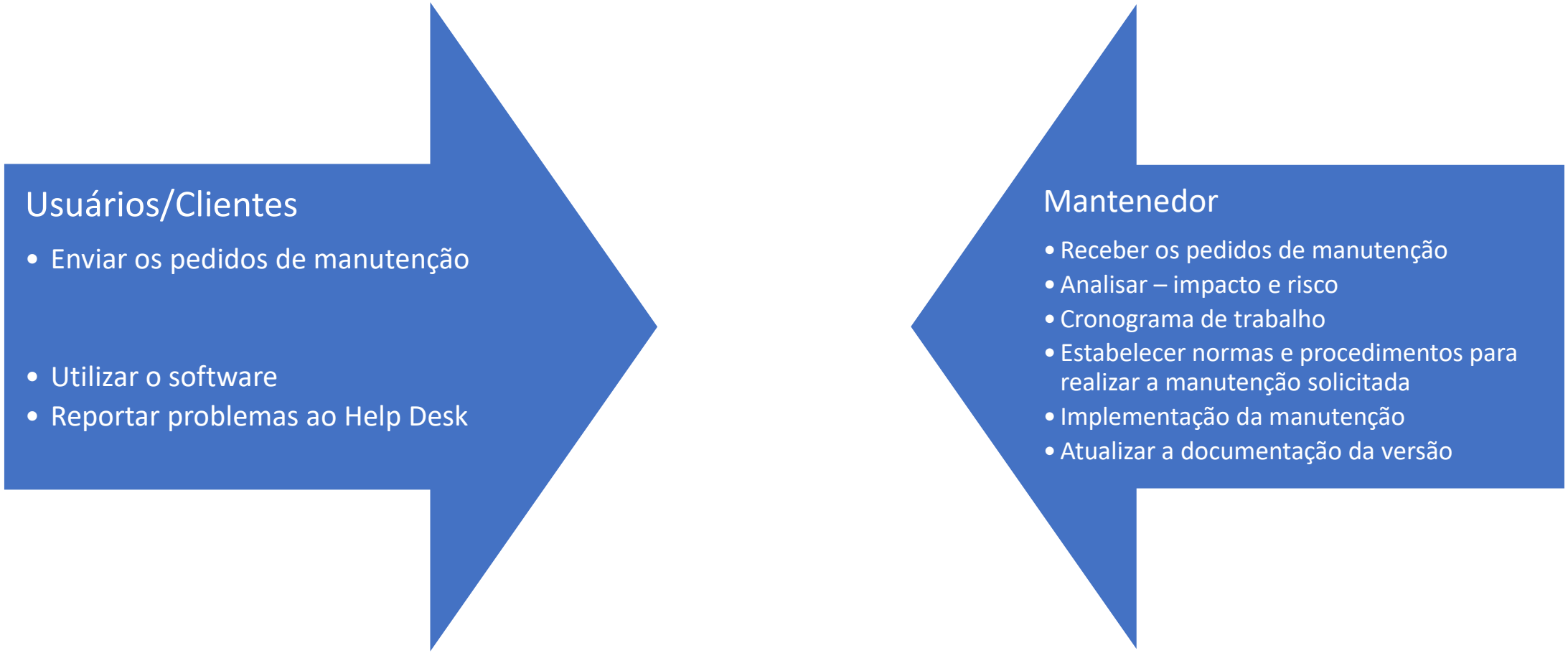
Atividades do Mantenedor

- Manter o controle das funções do software no dia a dia;
- Manter o controle sobre as modificações do software;
- Aperfeiçoar as funções existentes;
- Identificar ameaças a segurança e corrigir vulnerabilidades de segurança;
- Prevenir a degradação da estrutura do software devido a constantes mudanças.

Desafios da manutenção

- Problemas técnicos relacionadas a compreensão limitada
- Tempo curto exigido para realizar as modificações
  - Realizada por uma equipe diferente da que originou o software
- Documentação deficiente
  - Compreensível
  - Consistente com o código-fonte
- Dificuldade em rastrear a evolução do software
  - Muitas versões sem ajustes adequados na documentação

Estrutura da manutenção de software



Análise de impacto

- Analisar as solicitações de mudanças
- Replicar ou verificar o problema
- Desenvolver opções para implementar a modificação
- Elaborar documento de solicitação de mudança informando as opções de execução
- Obter a aprovação para a opção de modificação selecionada

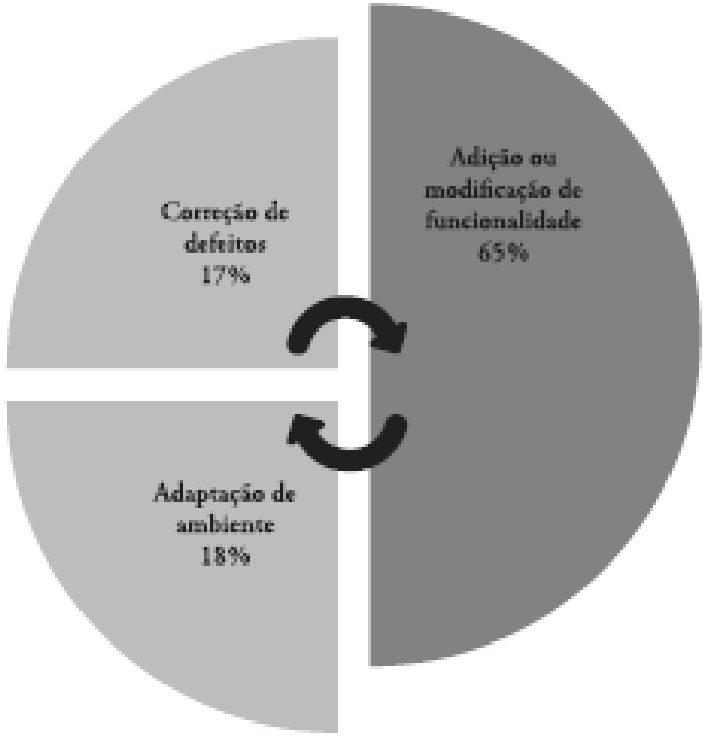
A gravidade de um problema é muitas vezes usada para decidir como e quando ocorrerá a correção.
O grau de impacto é muitas vezes usado para decidir se a alteração irá ocorrer ou não.

Tipos de manutenção

- Conforme a necessidade da manutenção:
  - Correção de defeitos ou manutenção corretiva
    - Correção de erros observados durante a fase de operação
    - Retrabalho da equipe de desenvolvimento gerando custos adicionais ao projeto
    - Os mais caros são os referentes à má interpretação dos requisitos
  - Adaptação ambiental ou manutenção adaptativa
    - Correções para que possa ser executado sobre o novo ambiente
      - Hardware, plataforma do sistema operacional ou alteração de um software de apoio
  - Adição de novas funcionalidades ou manutenção de aperfeiçoamento
    - Resposta às mudanças organizacionais e de negócio
    - Evolução do software para atender às necessidades do cliente/usuário

- Categoria de manutenção:
  - Proativa é preventiva e de aperfeiçoamento do software
  - Reativa é corretiva e adaptativa

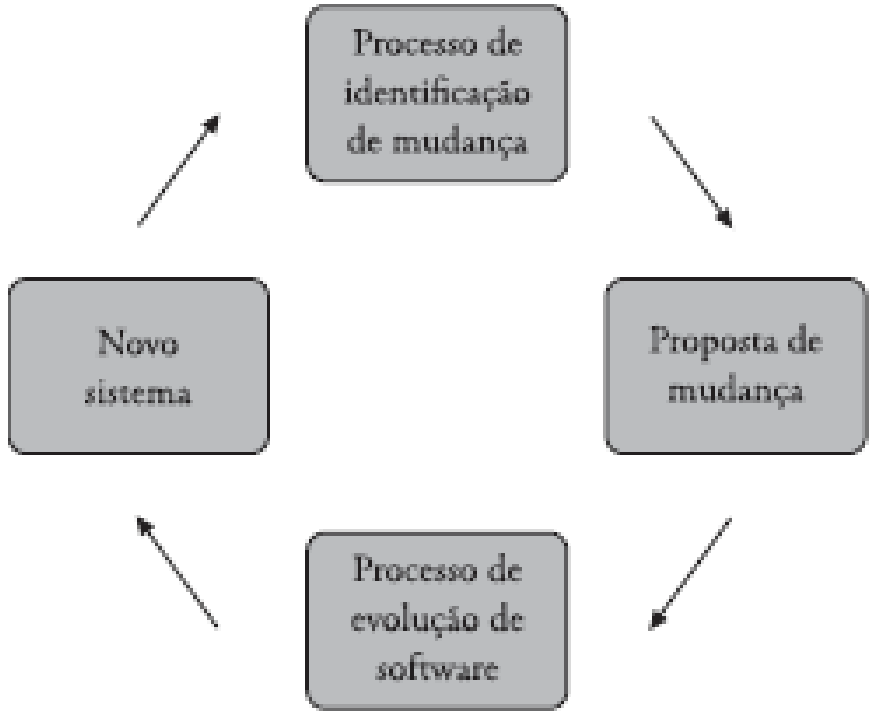
Tipos de manutenção



- Correção de defeitos ou manutenção corretiva – 17%
- Adaptação ambiental ou manutenção adaptativa – 18%
- Adição de novas funcionalidades ou manutenção de aperfeiçoamento – 65%

Processos de manutenção

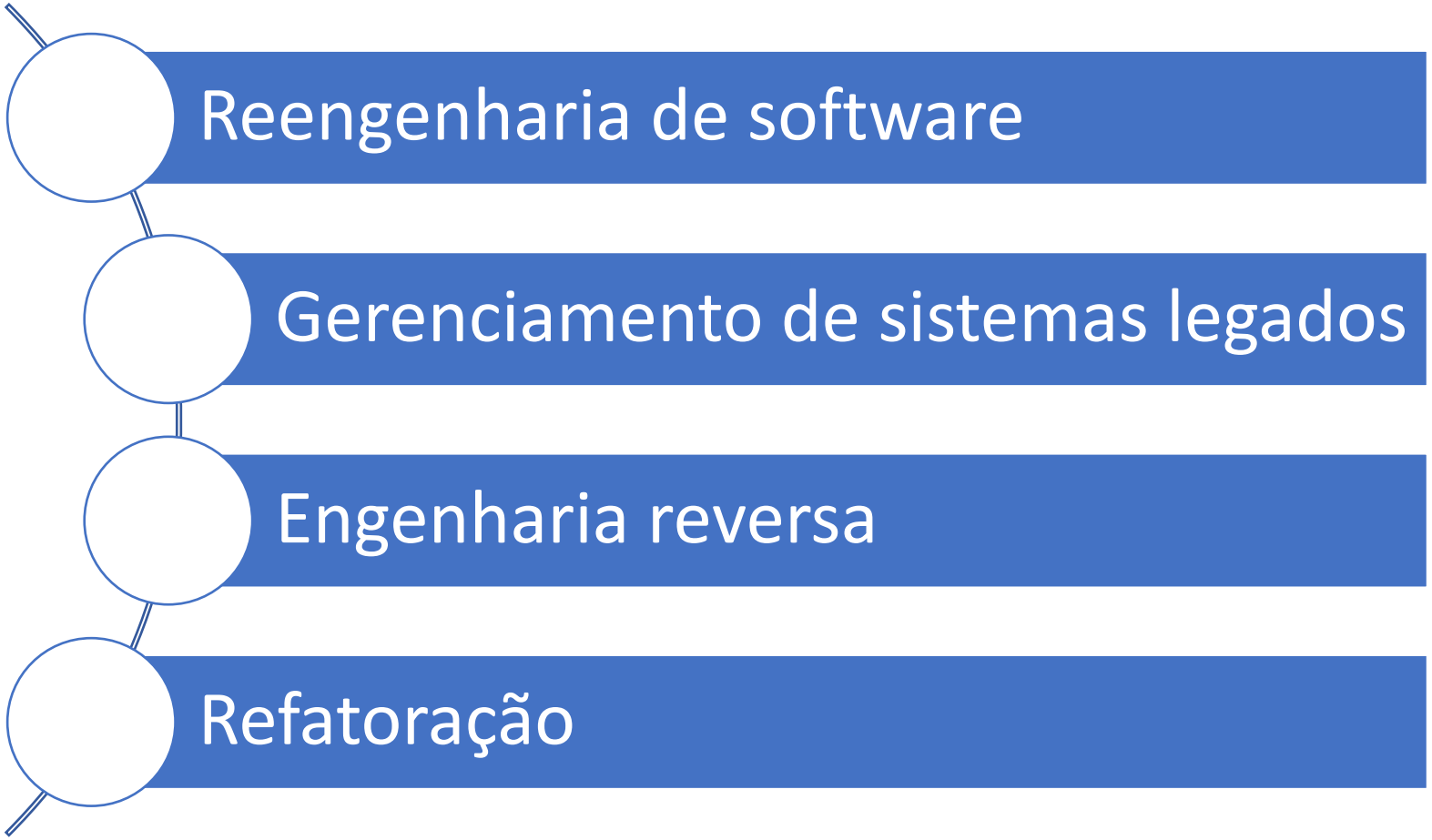
- São cíclicos e continuam durante todo o ciclo de desenvolvimento de vida de um sistema:
  - Requisitos existentes mas não implementados
  - Solicitação de novos requisitos
  - Relatórios de bugs do sistema apontado pelos usuários
  - Novas ideias para melhoria do sistema – evolução



Situações a serem gerenciadas

- Esforço de análise e adaptação
  - Código-fonte
  - Documentação associada
- Verificação
- Validação
- Adaptação
  - Impacto
  - Nova versão

Técnicas para manutenção



Técnicas para manutenção

- Reengenharia de software
  - **Reconstrução de um sistema a partir de sua documentação** e exame
    - Sem alterar as funcionalidades
    - Melhorando a qualidade
  - Em caso de sistemas legados e com documentação confusa
  - Pode ser de programas ou de dados

- Gerenciamento de sistemas legados
  - Precisam ser ampliados e adaptados à necessidade do negócio
  - Normalmente são **sistemas críticos**
    - Descartar – não contribui mais com o negócio
    - Manutenção regular – ainda é necessário e está estável
    - Reestruturar – desenvolver novos componentes
    - Substituir – não pode mais continuar operando

Técnicas para manutenção

- Engenharia reversa
  - Análise de software para
    - identificar os componentes e suas inter-relações e
    - criar representações do software sob outra forma ou
    - em níveis mais abstratos de representação.
  - **Analisar o sistema antigo com o objetivo de adaptá-lo à realidade atual**, adicionando novas funcionalidades para atender aos usuários.
  - Engenharia reversa é um subconjunto da reengenharia de software
  - Recuperação do projeto para auxiliar em manutenções futuras
  - É interessante após algum tempo de manutenções realizadas e com o aumento do custo da manutenção

Técnicas para manutenção

- Refatoração
  - **Processo de fazer melhorias em um programa** para diminuir a degradação gradual resultante de mudanças
  - Pode ser um processo contínuo de melhoria ao longo do processo de desenvolvimento e evolução do software para evitar a degradação do código, evitando aumentar os custos e dificuldade em futuras manutenções
  - Busca reduzir a complexidade e tornar o código mais compreensível.
  - “Maus cheiros” - *Code Smell* e *Bad Code Smell*
    - Código duplicado – ou semelhantes em diferentes partes do programa → único método com reuso
    - Métodos longos – transformar em métodos mais curtos
    - Declarações switch (case) – para evitar duplicação e depender de valor fixo → polimorfismo
    - Aglutinações dos dados – quando classes, parâmetros em métodos reiniciem de vários lugares do programa → encapsulamento
    - Generalidades especulativas – uso de generalidades para necessidades futuras → podem ser removidas

Atividade

- Realizar em grupo
- Cada grupo buscar em diferentes LLMs de IA Generativa
- Tema: formas possíveis de tornar o código do programa mais compreensível → evitar o *code smell*!
- Cada grupo deve buscar no mínimo 3 situações de *code smell*, identificando:
  - *Code Smell*
  - Benefícios de refatorar esse *code smell*
  - Como refatorar para melhorar o código
  - Exemplo em uma linguagem de programação – *code smell* e sua refatoração

- Postar nas tarefas do Teams até o final da aula de hoje!
- Incluir o nome dos integrantes do grupo no arquivo postado